

Projecto de Base de Dados

YOURMEDIA

Grupo 7
Prática 7



24/06/2022

Nuno Cunha - 98124
Filipe Silveira - 97981

Índice

Introdução	3
Requisitos Funcionais	4
Entidades	5
DER – Diagrama Entidade-Relação	6
ER – Esquema Relacional	7
SQL DML	8
Normalização	9
Triggers	10
Stored Procedures	11
User Defined Functions	13
Segurança	15
Conclusão	17

Introdução

Com a recente explosão em serviços de streaming online tais como, NETFLIX, HBO, Disney+ e muitos outros, consideramos que é um mercado em expansão e por isso consideramos importante que exista um sistema de avaliação do conteúdo que estes serviços oferecem.

YourMedia é um serviço de críticas online que permite aos utilizadores registados avaliarem filmes e séries numa pontuação de 0 a 10. Outra das funcionalidades é poderem criticar o filme e/ou adicioná-lo a uma das seguintes listas: avaliados ou ver mais tarde, assim como votarem nas críticas de outros utilizadores. Este serviço também oferece a possibilidade de ordenar os conteúdos pelos diversos critérios do conteúdo respectivo.

Por último, cada utilizador possui uma conta onde pode consultar os seus dados pessoais. A nossa base de dados é inspirada no IMDb, mas com algumas alterações para atender aos critérios de avaliação da disciplina.

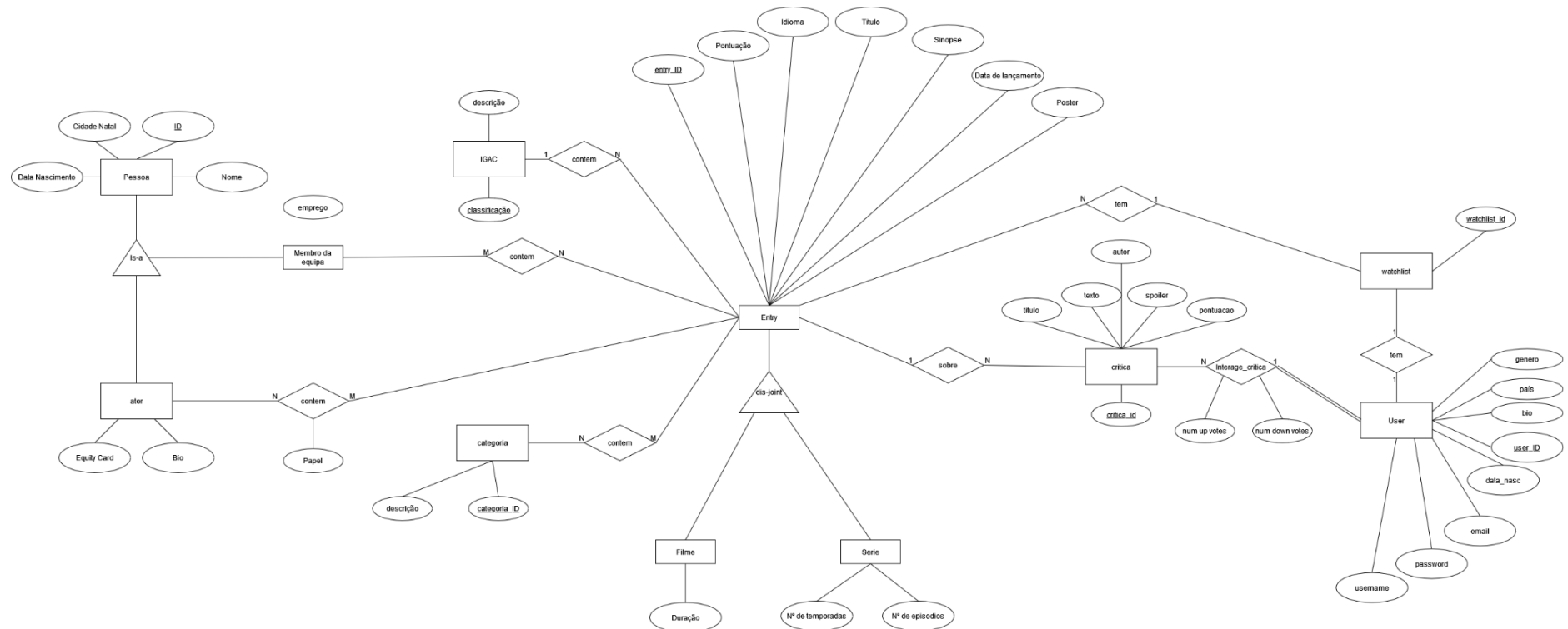
Requisitos Funcionais

- Avaliar filmes e séries
- Consultar dados pessoais
- Pesquisar por filmes/séries
- Marcar filme/série como avaliado ou ver mais tarde
- Criticar filme/série
- Ordenar por data de lançamento, nome e pontuação
- Votar em críticas de outros utilizadores

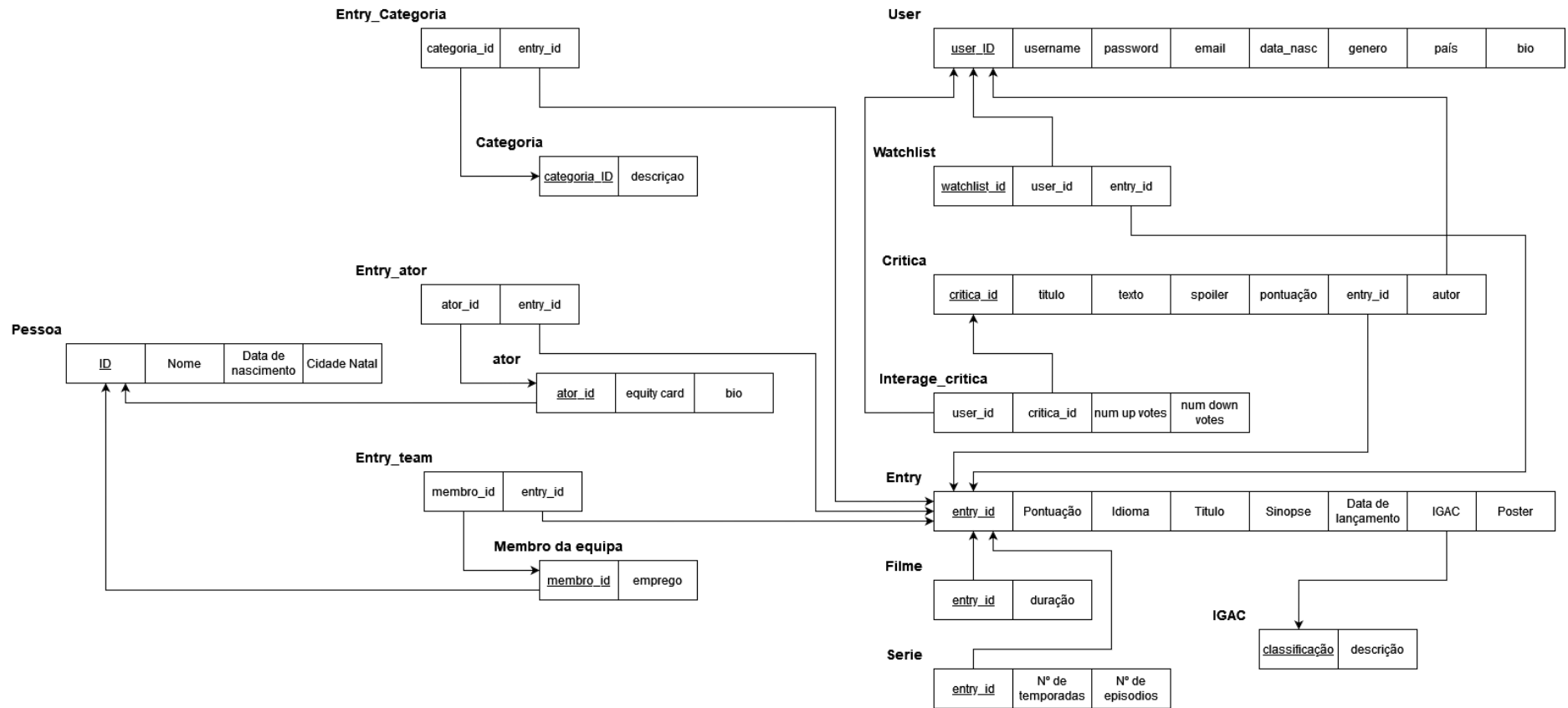
Entidades

- User - cliente da plataforma de críticas de filmes e séries online.
- Críticas - opiniões de utilizadores sobre um certo filme ou série.
- Filme - Obra cinematográfica com uma certa duração, para ser exibido no cinema, na televisão ou em algum outro serviço.
- Série - Obra cinematográfica dividida em episódios, cada um com a sua unidade, que são geralmente difundidos com intervalos regulares.
- Categoria - Conjunto de keywords que um filme/série pode ter.
- IGAC - Sistema de classificação de conteúdo para filmes/séries com base em sua adequação ao público.
- Elenco - lista do pessoal que participou no filme (ex. atores).
- Equipa - lista do pessoal que ajudou a produzir o filme (ex. realizador e produtor).
- Ator - Pessoa que representa uma personagem em filmes ou séries.
- Pessoa - Criatura humana caracterizada por um nome, data de nascimento, cidade natal e um ID.
- Ver mais tarde - Lista de filmes ou séries que o utilizador marcou como ver mais tarde

DER – Diagrama Entidade-Relação



ER – Esquema Relacional



SQL DML

Os comandos de SQL DML foram executados tanto na interface como dentro de Stored Procedures, UDFs e Triggers para facilitar os acessos e criar uma camada de abstração entre a interface e a Base de Dados.

Normalização

Após a análise da base de dados foram definidas as relações e as entidades necessárias para o funcionamento da mesma. Durante todo o processo foram respeitadas as regras impostas com o intuito de obter a Terceira Forma Normal (3FN), respeitando as formas normais anteriores.

Triggers

TriggerCritica – ao inserir uma critica, a pontuação desta critica irá afetar a classificação global do filme

```
GO
CREATE TRIGGER TriggerCritica ON Critica
AFTER INSERT
AS
    DECLARE @entryID INT
    DECLARE @critica_pontuacao REAL
    SELECT @entryID=entry_id_tb_critica,@critica_pontuacao=pontuacao
FROM inserted

    DECLARE @pontuacao REAL
    DECLARE @num_criticas INT
    SELECT @pontuacao=pontuacao,@num_criticas=num_criticas FROM
Media_Entry WHERE entry_ID=@entryID

    DECLARE @res REAL
    SET @res = ((@pontuacao * @num_criticas) + @critica_pontuacao) /
(@num_criticas + 1)

    UPDATE Media_Entry SET pontuacao = @res, num_criticas =
@num_criticas + 1 WHERE entry_ID=@entryID
GO
```

Stored Procedures

CheckIfMovie – Recebe o entry ID e retorna se este é um filme(TRUE) ou uma serie(FALSE)

InsertCritica – Cria uma critica para um certo filme/serie

DeleteCriticaByCriticalID – Elimina uma critica de um certo filme/serie

UpvoteCriticaByCriticalID – Incrementa o número de upvotes de uma certa critica

DownvoteCriticaByCriticalID - Incrementa o número de downvotes de uma certa critica

AddWatchlistByEntryID – Adiciona um filme/serie à lista para ver mais tarde do utilizador

DeleteWatchlistByWatchID - Remove um filme/serie pelo WatchID da lista para ver mais tarde do utilizador

DeleteWatchlistByEntryIDuserID - Remove um filme/serie pelo EntryID e userID da lista para ver mais tarde do utilizador

CheckIfInWatchlist – Verifica se o filme/serie esta na lista para ver mais tarde do utilizador

DoLogin – Realiza o login do utilizador (com encriptação) e retorna o id do utilizador se for realizado com sucesso

DoRegister – Realiza o registo de um novo utilizador (com encriptação) e verifica que não há colisões na base de dados

```
CREATE PROCEDURE CheckIfInWatchlist @userID INT, @entry_id INT
AS
    RETURN ( SELECT COUNT(1) FROM Watchlist WHERE
username_ID_tb_watchlist=@userID AND entry_ID_tb_watchlist=@entry_id )
GO
```

```
CREATE PROCEDURE InsertCritica @titulo VARCHAR(100), @texto
VARCHAR(5000), @spoiler BIT, @pontuacao REAL, @entry_id INT, @autor INT
AS
    INSERT INTO Critica VALUES
(@titulo,@texto,@spoiler,@pontuacao,@entry_id,@autor)
GO
```

```

CREATE PROCEDURE DoLogin @email VARCHAR(250), @pwd VARCHAR(250)
WITH ENCRYPTION
AS

    IF EXISTS( SELECT username_ID FROM Username WHERE email=@email AND
user_password= HASHBYTES('SHA2_256',@pwd) )
        BEGIN
            DECLARE @userID INT
            SELECT @userID = username_ID FROM Username WHERE
email=@email AND user_password= HASHBYTES('SHA2_256',@pwd)
            RETURN @userID
        END

    RETURN -1

GO

```

```

CREATE PROCEDURE DoRegister @username VARCHAR(250),
                            @pwd VARCHAR(250),
                            @email VARCHAR(250),
                            @data_nasc DATE,
                            @genero VARCHAR(1),
                            @pais VARCHAR(100),
                            @bio VARCHAR(5000)
WITH ENCRYPTION
AS

    IF NOT EXISTS( SELECT username_ID FROM Username WHERE email=@email
OR username=@username)
        BEGIN
            INSERT INTO Username VALUES (@username,
                                         HASHBYTES('SHA2_256',@pwd),
                                         @email,
                                         @data_nasc,
                                         @genero,
                                         @pais,
                                         @bio)

            RETURN 1
        END

    RETURN 0

GO

```

User Defined Functions

ShowActorByEntryId – Devolve todas as informações dos atores de um certo filme/serie

ShowTeamByEntryId - Devolve todas as informações da equipa de um certo filme/serie

ShowMovieByEntryId – Devolve todas as informações sobre um filme

ShowSerieByEntryId - Devolve todas as informações sobre uma Serie

ShowCriticaByEntryID – Devolve todas as criticas de um filme/serie

ShowVotesByCriticalID – Devolve o número total de upvotes e downvotes de uma critica

ShowCriticaByUserID – Devolve todas as criticas de um determinado utilizador

ShowWatchlistByUserID – Devolve todos os filmes/series da lista ver mais tarde do utilizador respetivo

ShowInfoActorByActorId – Devolve todas as informações sobre o ator respetivo

ShowWhereCastByActorId – Devolve todas os filmes/series onde o ator já participou

GetCategoriaByEntryId(auxiliar) – Devolve todas as categorias de um certo filme/serie

```
CREATE FUNCTION ShowActorByEntryId(@entry_id INT) RETURNS Table
AS
    RETURN (SELECT * FROM Pessoa JOIN (SELECT * FROM Entry_Ator JOIN
Ator ON Entry_Ator.atorID=Ator.tb_atorID WHERE
Entry_Ator.entryID_tb_ator=@entry_id) AS tmp ON
Pessoa.ID=tmp.tb_atorID)
GO
```

```
CREATE FUNCTION ShowMovieByEntryId(@entry_id INT) RETURNS Table
AS
    RETURN (SELECT * FROM IGAC JOIN (SELECT * FROM
GetCategoriaByEntryId(@entry_id) AS tmp_categoria JOIN (SELECT * FROM
Media_Entry JOIN Filme ON Media_Entry.entry_ID=Filme.entry_ID_tb_filme
WHERE Media_Entry.entry_ID=@entry_id) AS tmp ON
tmp_categoria.entryID_tb_categoria=tmp.entry_ID) AS tmp2 ON
IGAC.classificacao=tmp2.IGAC )
GO
```

```
CREATE FUNCTION ShowVotesByCriticaID(@CriticaID INT) RETURNS Table
AS
    RETURN (SELECT SUM(num_up_votes) AS
num_up_votes,SUM(num_down_votes) AS num_down_votes FROM
Interage_critica WHERE
Interage_critica.critica_ID_tb_interage=@CriticaID)
GO
```

```
CREATE FUNCTION ShowWatchlistByUserID(@userID INT) RETURNS Table
AS
    RETURN (SELECT * FROM Media_Entry JOIN (SELECT
entry_ID_tb_watchlist FROM Watchlist JOIN Username ON
Watchlist.username_ID_tb_watchlist=Username.username_ID WHERE
Username.username_ID=@userID) AS tmp ON
Media_Entry.entry_ID=tmp.entry_ID_tb_watchlist )
GO
```

Segurança

No nosso projeto tivemos em atenção o aspecto de segurança, em aspetos como o login e o registo. Onde tanto a função como a password é encriptada.

```
CREATE PROCEDURE DoLogin @email VARCHAR(250), @pwd VARCHAR(250)
WITH ENCRYPTION
AS

    IF EXISTS( SELECT username_ID FROM Username WHERE email=@email AND
user_password= HASHBYTES('SHA2_256',@pwd) )
        BEGIN
            DECLARE @userID INT
            SELECT @userID = username_ID FROM Username WHERE
email=@email AND user_password= HASHBYTES('SHA2_256',@pwd)
            RETURN @userID
        END

    RETURN -1

GO
```

```

CREATE PROCEDURE DoRegister @username VARCHAR(250),
                           @pwd VARCHAR(250),
                           @email VARCHAR(250),
                           @data_nasc DATE,
                           @genero VARCHAR(1),
                           @pais VARCHAR(100),
                           @bio VARCHAR(5000)
WITH ENCRYPTION
AS
    IF NOT EXISTS( SELECT username_ID FROM Username WHERE email=@email
OR username=@username)
    BEGIN
        INSERT INTO Username VALUES (@username,
                                     HASHBYTES('SHA2_256',@pwd),
                                     @email,
                                     @data_nasc,
                                     @genero,
                                     @pais,
                                     @bio)

        RETURN 1
    END

    RETURN 0
GO

```

Conclusão

Com este projeto pudemos ver na prática o funcionamento e a utilidade de um Sistema de Gestão de Base Dados, bem como a importância de ter um local centralizado da informação para permitir a consistência e segurança dos dados.

Apesar de não ser a parte central do trabalho, o facto de ter uma interface intuitiva permitiu que a visualização dos dados e a interação com este fosse mais fluída.

Concluindo, os objetivos propostos no início do projeto foram alcançados, permitindo assim o aprofundamento do conhecimento relacionado à matéria lecionada na cadeira de Base de Dados.